

COMPANION MODULE

Getting Started with Claude Code & Claude Cowork

A hands-on beginner's guide to Anthropic's two ways of putting an AI agent to work for you — the no-code Cowork desktop app and the developer-focused Claude Code — taught side by side from absolute zero.

Format: Self-paced · two parallel tracks · video + slides + readings

Structure: Shared intro + 5 Cowork modules + 5 Claude Code modules + reference

Level: Total beginner — no prior AI or coding experience assumed

Companion to: *AI Agents & Workflow Automation* (the 12-week certificate program)

Information verified against Anthropic's official docs, June 2026

About this companion module

The main course teaches you **what AI agents are and how they think**. This companion module is where you actually **put one to work**. Anthropic offers two front doors to the same underlying agent technology, and this module teaches both, in equal depth, from scratch:

- **Claude Cowork** — a no-code desktop app. You describe an outcome in plain English and Claude does the work directly with your files. Built for non-technical knowledge workers.
- **Claude Code** — a developer tool you run mostly in the "terminal." Same agent, aimed at people writing and managing software.

Both are built on the **same agentic architecture** — in fact, Cowork is described by Anthropic as bringing "Claude Code's agentic capabilities" to the desktop without the terminal. Learning both means you'll understand the full range, and can pick the right door for any job.

How this module is organized

Because Cowork is no-code and Claude Code is developer-focused, each gets its own self-contained track of five modules. They're deliberately mirrored so you can compare them at each stage.

- **Shared Foundations** — what these tools are, plans, and accounts (read this first).
- **Track A — Claude Cowork** (5 modules): install & setup → first task → safety & permissions → skills, connectors, plugins → projects & automation.
- **Track B — Claude Code** (5 modules): install & terminal basics → first session → making changes safely → everyday workflows → customizing.
- **Reference** — "which tool when," command cheat sheets, and a glossary.

A NOTE ON ACCURACY

Claude Code and Cowork are evolving products. This curriculum reflects Anthropic's official documentation as of June 2026. When you build the lessons, point learners to code.claude.com/docs and support.claude.com for the latest screens and steps, since menus and buttons may shift over time.

READ FIRST

Shared Foundations

Before either track

Five minutes of orientation that applies to both tools: what "agentic" means in practice, which Claude plan you need, and how the two tools relate.

s

One agent, two front doors

WHAT YOU'LL LEARN

- The difference between **regular chat** (Claude answers) and an **agent** (Claude does the work — reads files, runs steps, produces finished output).
- That Cowork and Claude Code are **the same engine** with different interfaces — Cowork for non-coders on the desktop, Claude Code for developers in the terminal and editors.
- What you need to begin: a **paid Claude plan** (Pro, Max, Team, or Enterprise) and the right app installed.

KEY FACTS TO TEACH

Question	Answer
Who is Cowork for?	Anyone doing knowledge work — no coding. Runs in the Claude Desktop app (macOS & Windows).
Who is Claude Code for?	People working with software/code. Runs in the terminal, plus web, desktop, VS Code, JetBrains, and Slack.
What plan do I need?	A paid plan — Pro, Max, Team, or Enterprise. (Claude Code also supports Console/API and enterprise cloud.)
Do they share an engine?	Yes — Cowork brings Claude Code's agentic capabilities to the desktop without the terminal.

ANALOGY

Think of one powerful **engine** that fits in two different vehicles. **Cowork** is the automatic car — get in, say where you want to go. **Claude Code** is the manual car with developer controls — more direct command over the machinery, in exchange for knowing how to drive a stick. Same engine underneath.

ON-SCREEN VISUAL

A single glowing "agent engine" in the center, with two arrows out to two device frames: a friendly desktop-app window labeled "Cowork (no code)" and a dark terminal window labeled "Claude Code (developer)."

TRACK A

Claude Cowork — the no-code track

5 modules · zero coding

Cowork lets you describe an outcome in plain English and have Claude complete real, multi-step work with your files — formatted documents, organized folders, research, spreadsheets, and slides — all from the Claude Desktop app.

OBJECTIVES

- Confirm your computer can run Cowork and that you have a paid plan.
- Install the Claude Desktop app (macOS or Windows) and switch into Cowork.
- Understand that Cowork is desktop-only (not web), and the app must stay open while it works.

LESSONS

- **A1.1 — Requirements check.** A paid plan (Pro/Max/Team/Enterprise), the desktop app, and a live internet connection. There's a "readiness check" download that confirms your machine is compatible.
- **A1.2 — Install the desktop app.** Download from claude.com/download, install, and sign in.
- **A1.3 — Find Cowork.** In the app, use the mode selector between "Chat" and "Cowork," then click the Cowork tab to switch to "Tasks."
- **A1.4 — Keep it open.** The app must stay open and the computer awake for Claude to keep working.

ANALOGY

Switching from Chat to Cowork is like moving from **texting an assistant for advice** to **handing them the keys to your desk** so they can actually do the filing. The Chat/Cowork toggle is that doorway.

ON-SCREEN VISUAL

A labeled screenshot of the Claude Desktop window with a red circle around the "Chat | Cowork" mode toggle and an arrow pointing to the "Tasks" view.

TRY IT

Run the readiness check, install the app, sign in, and click into the Cowork tab. You don't have to do anything yet — just get to the "describe a task" screen.

OBJECTIVES

- Write a clear task description and let Claude plan and execute it.
- Connect a folder so Claude can read and write your actual files.
- Read the progress indicators and steer mid-task when needed.

LESSONS

- **A2.1 — Describe the outcome, not the clicks.** e.g., "Organize my Downloads folder by type and date" or "Turn these meeting notes into a one-page summary."
- **A2.2 — Connect a folder.** Claude can only touch folders you choose; finished files are delivered right back to your file system.
- **A2.3 — Watch it work.** Progress indicators and visible reasoning let you follow along; you can jump in to course-correct.
- **A2.4 — Editing drafts in place.** For Markdown drafts, highlight text and click "Edit with Claude" to revise exactly that spot.

ANALOGY

Connecting a folder is like deciding **which drawers the assistant may open**. They can't rummage through the whole house — only the drawers you unlock.

ON-SCREEN VISUAL

Before/after of a messy "Downloads" folder turning into tidy, dated subfolders, with a side panel showing Claude's live step list ("Scanning files → Grouping by type → Creating folders → Moving files").

TRY IT

Connect a safe test folder with a few sample files and ask Claude to organize or summarize them. Watch the plan before letting it run.

OBJECTIVES

- Understand the two permission modes and when to use each.
- Know the built-in protections: isolated code execution, folder-limited access, and delete protection.
- Build the habit of reviewing Claude's plan before approving, especially with sensitive files.

LESSONS

- **A3.1 — Two modes.** "Ask before acting" (Claude pauses for your approval on each step — recommended for beginners) vs. "Act without asking" (faster, riskier — only when supervising trusted work).
- **A3.2 — Built-in guardrails.** Code runs in an isolated virtual machine separate from your operating system; Claude can only read/write folders you connected; network access follows your settings.
- **A3.3 — Delete protection.** Claude always asks for explicit permission ("Allow") before permanently deleting any file, in both modes.
- **A3.4 — Review before you trust.** Cowork has internet access and takes real actions — glance at the plan first.

ANALOGY

"Ask before acting" is a **learner driver with an instructor's brake** — every move gets a nod first. The isolated VM is a **sandbox**: Claude can build and knock things over in the sandbox without harming the rest of your computer.

ON-SCREEN VISUAL

A toggle graphic: "Ask before acting ☐ recommended" vs. "Act without asking ☒ supervise." Below it, a permission pop-up reading "Allow Claude to delete these files?" with Allow / Deny buttons.

OBJECTIVES

- Tell apart the three ways to extend Cowork: skills, connectors, and plugins.
- Understand how each makes Claude more capable for your specific work.
- Know where to browse and add them.

LESSONS

- **A4.1 — Skills.** A skill is plain-text instructions that teach Claude how to do one specific kind of task; it loads only when relevant and can be shared with teammates.
- **A4.2 — Connectors.** These link Claude to your apps and services so it can fetch data and take actions — e.g., search Google Drive, send a Slack message, create a Linear issue.
- **A4.3 — Plugins.** Bundles that package skills, connectors, and sub-agents together, tailored to a role, team, or company.
- **A4.4 — Global & folder instructions.** Standing instructions (tone, format, your role) that apply to every session or to a specific folder.

Extension	What it is	Everyday example
Skill	A reusable "how-to" for one task	"Format every report in our house style"
Connector	A link to an outside app	Search your Google Drive; post to Slack
Plugin	A bundle of skills + connectors + sub-agents	A ready-made "marketing team" pack

ANALOGY

A **skill** is teaching your assistant one recipe. A **connector** is giving them a key to another building (your Drive, your Slack). A **plugin** is hiring a pre-trained specialist who already knows several recipes and has all the right keys.

ON-SCREEN VISUAL

Three stacked cards — "Skill (a recipe card)," "Connector (a key/plug icon to app logos)," and "Plugin (a box containing both)" — showing how a plugin wraps the other two.

OBJECTIVES

- Use Projects to keep related work, files, and memory together.
- Set tasks to run automatically on a schedule.
- Know what professional outputs Cowork can produce, and how to assign tasks from your phone.

LESSONS

- **A5.1 — Projects.** Persistent workspaces with their own files, instructions, and memory — ideal for recurring or long-running work. (Memory is retained within a project.)
- **A5.2 — Scheduled tasks.** Type `/schedule` in a task, or use the "Scheduled" sidebar, to have Claude run work automatically (e.g., a Monday-morning summary). Runs only while your computer is awake and the app is open.
- **A5.3 — Professional outputs.** Excel files with working formulas, PowerPoint decks, and formatted documents — not just rough CSVs.
- **A5.4 — From anywhere.** Pro and Max users can message Claude from the mobile app while the desktop keeps working, and get results back.

ANALOGY

A **Project** is a dedicated **desk with its own filing cabinet** for one client or initiative — everything stays together and Claude "remembers" the context. A **scheduled task** is a **standing appointment** on Claude's calendar.

ON-SCREEN VISUAL

A sidebar mock with a "Projects" list and a "Scheduled" section; a small calendar badge shows "Every Monday 8:00 AM — Weekly report." Output icons (Excel, PowerPoint, Word) sit beside a finished deliverable.

TRY IT

Create a Project, drop in a couple of files, and schedule a simple recurring task such as a weekly summary of a folder.

TRACK B

Claude Code — the developer track

5 modules · gentle intro to the terminal

Claude Code is the same agent for people who work with software. It lives mainly in the "terminal" but also runs in VS Code, JetBrains, the web, and Slack. This track assumes **no coding background** and explains every term as it appears.

OBJECTIVES

- Understand what the "terminal" is and lose the fear of it.
- Install Claude Code and sign in to your account.
- Know that Claude Code also runs in editors, the web, and Slack — not only the terminal.

LESSONS

- **B1.1 — What's a terminal?** A plain text window where you type commands instead of clicking. It looks intimidating but you'll only need a few commands.
- **B1.2 — Install it.** The recommended native installer is a single command — macOS/Linux: `curl -fsSL https://claude.ai/install.sh | bash`; Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`. (Homebrew, WinGet, and npm are alternatives.)
- **B1.3 — Log in.** Type `claude` to start; on first run it opens your browser to sign in. Use `/login` later to switch accounts.
- **B1.4 — It's everywhere.** Same tool in VS Code, JetBrains IDEs, the web (claude.ai/code), a desktop app, and Slack.

ANALOGY

The terminal is just a **plain-text doorway** into your computer — like sending instructions by note instead of pointing at buttons. Scary-looking, but it's only a notepad that listens.

ON-SCREEN VISUAL

A friendly annotated terminal window: a label points at the blinking cursor ("you type here"), and a callout shows the one install command with a green "press Enter" tag.

TRY IT

Open your terminal, paste the install command for your system, then type `claude` and sign in. That's the whole setup.

OBJECTIVES

- Start Claude Code inside a project folder.
- Ask plain-English questions about a codebase before changing anything.
- Understand that Claude reads project files automatically as needed.

LESSONS

- **B2.1 — Open a project.** `cd` into a folder, type `claude`, and you're in. The prompt shows your model and working directory.
- **B2.2 — Explore first.** Ask "what does this project do?", "what technologies does it use?", "where is the main entry point?" — Claude analyzes and explains.
- **B2.3 — No manual context.** You don't paste files in; Claude reads what it needs on its own.
- **B2.4 — Get help anytime.** Type `/help` for commands, or just ask "what can Claude Code do?"

ANALOGY

Like a **new employee on day one**, the smart first move is to *read the manual and look around* before touching anything. "Explore first" is Claude reading itself into the project.

ON-SCREEN VISUAL

A terminal showing the user typing `what does this project do?` and Claude returning a tidy plain-English summary of the project's purpose and structure.

OBJECTIVES

- Ask Claude to make a real change and review it before it's applied.
- Understand approval: Claude always asks before modifying files.
- Know the permission modes and how to cycle them.

LESSONS

- **B3.1 — A first edit.** e.g., "add a hello world function to the main file." Claude finds the file, **shows the proposed change**, and asks for approval before editing.
- **B3.2 — Approve or accept all.** Approve changes one at a time, or enable "Accept all" for a session when you trust the work.
- **B3.3 — Permission modes.** Press `Shift+Tab` to cycle modes (how much it pauses for approval) — the same "ask vs. act" trade-off you saw in Cowork.
- **B3.4 — Be specific.** "Fix the login bug where users see a blank screen after wrong credentials" beats "fix the bug."

ANALOGY

Claude showing a change before applying it is like **"tracked changes" in a Word document** — you see the proposed edit highlighted and click accept or reject. Nothing changes without your nod.

ON-SCREEN VISUAL

A diff view in the terminal: removed lines in red, added lines in green, with an approval prompt "Apply this change? (y/n)" highlighted.

TRY IT

In a throwaway practice project, ask Claude to "add a comment at the top of the main file explaining what it does," then approve the change.

OBJECTIVES

- Use Claude for common developer tasks in plain English.
- Understand, simply, what Git does and how Claude makes it conversational.
- Break bigger requests into clear steps.

LESSONS

- **B4.1 — Git, made conversational.** Git is a "save-history" system for code. Ask "what files have I changed?", "commit my changes with a descriptive message," or "create a branch called feature/login."
- **B4.2 — Fix bugs & add features.** Describe the problem or feature in words; Claude locates the code, implements a fix, and runs tests if available.
- **B4.3 — Tests, docs, refactors.** "Write unit tests for the calculator functions," "update the README with installation instructions," "review my changes and suggest improvements."
- **B4.4 — Step-by-step prompts.** For larger jobs, number the steps so Claude tackles them in order.

ANALOGY

Git is a **video-game save system** for your project: checkpoints you can return to. A "commit" is hitting Save with a note about what changed — and Claude can write that note for you.

ON-SCREEN VISUAL

A simple timeline of code "save points" (commits), each with a label; an arrow shows branching off to a "feature/login" lane and merging back.

OBJECTIVES

- Create a CLAUDE.md so Claude remembers your project's rules.
- Understand slash commands, skills, hooks, and MCP at a beginner level.
- Know the handful of commands you'll use daily.

LESSONS

- **B5.1 — CLAUDE.md.** A plain-text file Claude reads at the start of every session — your project's standing instructions and conventions. The fastest way to create one is the `/init` command, which scans your project and drafts it.
- **B5.2 — Slash commands & skills.** Type `/` to see commands and skills; skills teach Claude reusable, task-specific know-how (the same idea as Cowork skills).
- **B5.3 — Hooks & MCP, briefly.** Hooks run your own actions automatically at certain points; MCP connects Claude to outside tools and data — the developer cousin of Cowork's connectors.
- **B5.4 — Daily commands.** A short cheat sheet (also in the Reference section).

Command	What it does
<code>claude</code>	Start an interactive session
<code>claude "task"</code>	Run a one-time task
<code>claude -c</code>	Continue your most recent conversation
<code>/init</code>	Generate a starter CLAUDE.md
<code>/help</code>	Show available commands
<code>/clear</code>	Clear the conversation history
<code>/exit</code> or Ctrl+D	Exit Claude Code

ANALOGY

CLAUDE.md is the **onboarding handbook** you hand a new hire — "here's how we do things." Write it once, and every future session already knows your rules.

ON-SCREEN VISUAL

A split: left shows running `/init` ; right shows the generated CLAUDE.md with sections (Build commands, Conventions, Notes) appearing.

REFERENCE

Which tool when, cheat sheets & glossary

Keep this handy

A quick decision guide, the commands learners reach for most, and plain-English definitions of every term used in this module.

Which tool should I use?

If you want to...	Use
Organize files, write reports, build spreadsheets/slides, research — without coding	Cowork
Have work delivered as polished Office documents on your desktop	Cowork
Schedule recurring knowledge-work tasks	Cowork
Write, fix, refactor, or test software code	Claude Code
Work inside a code editor (VS Code / JetBrains) or with Git	Claude Code
Automate parts of a software pipeline (CI/CD, Slack)	Claude Code

Rule of thumb: **files and documents** → **Cowork**; **source code** → **Claude Code**. Both use the same agent, so the skills you learn transfer.

Cowork quick reference

Action	How
Enter Cowork	Claude Desktop → mode toggle → "Cowork" → Tasks
Give file access	Connect a folder; Claude reads/writes only there
Choose safety level	"Ask before acting" (recommended) vs. "Act without asking"
Schedule work	Type <code>/schedule</code> or use the "Scheduled" sidebar
Group related work	Create a Project (keeps files, instructions, memory)
Set standing rules	Settings → Cowork → Global instructions

Claude Code quick reference

Command	What it does
<code>claude</code>	Start an interactive session
<code>claude -p "query"</code>	Ask one question, then exit
<code>claude -c</code> / <code>claude -r</code>	Continue / resume a conversation
<code>/init</code>	Create a starter CLAUDE.md
<code>/login</code>	Switch or re-authenticate your account
<code>Shift+Tab</code>	Cycle permission modes
<code>/help</code> · <code>/clear</code> · <code>/exit</code>	Help · clear history · quit

Plain-English glossary

Term	Meaning
Agentic	Claude doing multi-step work for you, not just answering a question.
Cowork	No-code agent mode inside the Claude Desktop app for knowledge work.
Claude Code	The agent for software work, used mainly in the terminal and code editors.
Terminal	A text window where you type commands instead of clicking.
Permission mode	How often Claude pauses to get your approval before acting.
Virtual machine (VM)	An isolated "sandbox" where Claude runs code safely, separate from your real system.
Connect a folder	Granting Cowork access to read/write a specific folder's files.
Skill	Reusable plain-text instructions teaching Claude one type of task.
Connector / MCP	A link letting Claude reach an outside app or data source (Cowork calls it a connector; the underlying tech is MCP).
Plugin	A bundle of skills, connectors, and sub-agents for a role or team.
Project (Cowork)	A persistent workspace with its own files, instructions, and memory.
Scheduled task	Work Claude runs automatically on a cadence you set.
CLAUDE.md	A file of standing instructions Claude Code reads at the start of every session.

Git	A "save-history" system for code; Claude Code can drive it conversationally.
Commit	A saved checkpoint of code changes, with a note describing them.

How this fits your course

Sell or bundle this as a standalone beginner module alongside the 12-week certificate. Each module's objectives, analogy, on-screen visual, and "Try it" boxes are the blueprint for its video script and slides — built straight from Anthropic's official documentation so learners get accurate, current guidance.